agent framework

1.架构选择

A1 的核心是“假说-验证-修正”循环，LangGraph 原生支持这种循环图（Cyclic Graph） 结构，能精确控制 Agent 在什么条件下回到“修改代码”节点。

- **节点 (Nodes)：** 设立“分析代理”、“漏洞编写代理”、“执行环境（沙盒）”等节点。
- **边 (Edges)：** 当沙盒反馈“执行失败”时，通过一条条件边将错误信息传回“编写代理”进行 Refine。
- **状态存储：** 自动保存之前的尝试记录，避免 Agent 犯同样的错误。

维度	LangGraph (LangChain 生态)	CrewAI	AutoGen
设计逻辑	基于状态机/图，精准控制流程	基于角色和任务分配 (Crew)	基于 Agent 间的对话和协作
循环能力	**极强**，原生支持复杂循环	较强，主要通过 Sequential 任务	强，通过对话反复修正
学习曲线	较陡（需要理解图逻辑）	简单（配置角色和任务即可）	中等
最佳用途	**复杂的工业级闭环安全审计系统**	快速原型或模拟团队协作流程	需要 Agent 频繁交流、自主运行代码的场景

模拟与评估：

- **Foundry (Anvil/Cast):** 用于实现 Local Forked Environment
- **Slither/Aderyn:** 用于初步的静态代码分析

对比

核心维度	LangGraph	CrewAI	AutoGen	Plain LangChain (传统链)
1. 循环能力 (Cyclic Capability) *(A1 核心: "报错→修改→重试")*	**原生支持 (Native)** 专为循环图设计，可以轻松定义“失败则跳回 Node 2”的条件边。	**弱 (Weak)** 主要是线性的 Process，实现复杂的“条件回滚”比较别扭，更适合单向任务。	**基于对话 (Conversational)** 靠 Agent 互相说话来循环，流程不可控，容易陷入死循环。	**不支持 (None)** DAG (有向无环图)，只能一条路走到黑，无法实现 A1 的迭代闭环。
2. 状态管理 (State Management)	**全局共享状态 (Shared State)**	**上下文传递 (Context)**	**对话历史 (History)**	**内存 (Memory)** 简单的键值对存

(A1 需传递: 源码, 报错, 尝试次数)	所有节点读写同一个 Schema，像传接力棒一样传递数据，数据结构极其清晰。	基于文本的 Context 传递，对于结构化数据（如漏洞对象、代码文件）的管理较弱。	所有状态都混在聊天记录里，随着对话变长，Token 消耗巨大且容易丢失重点。	储，难以处理复杂的工程状态流转。
3. 执行确定性 (Determinism) *(A1 需: 严格控制重试 5 次)*	**高 (High)** 你可以写 Python 代码精确控制逻辑（`if retries > 5: return End`）。	**中 (Medium)** 更偏向于让 Agent 自己决定，容易“跑偏”。	**低 (Low)** Agent 极其自主，很难强制它“严格按照步骤 1-2-3 执行”。	**高 (High)** 非常确定，但太死板，缺乏 A1 需要的动态调整能力。
4. 工具集成 (Foundry/Slither) *(A1 的手)*	**节点级控制 (Node-level)** 可以在特定的 Node 完美封装 Python 函数调用，易于调试。	**Tool 抽象** 工具集成容易，但很难控制 Agent 在**什么时候**必须用**哪个**工具。	**代码执行器** 强项是写代码运行，但在集成特定安全工具流时，不如图结构清晰。	**Tool调用** 支持，但缺乏上下文感知的复杂调用逻辑。
结论	✅ **最佳方案 (Best Fit)** 完美复现 A1 的“状态机”本质。	❌ **不推荐** 适合“模拟人类团队开会”，不适合“精密的代码工程”。	⚠️ **可选但不可控** 适合“探索性实验”，但在生产环境难以落地。	❌ **无法实现** 无法处理闭环逻辑。

2. dataset集成到agent中

- **RAG 增强 (知识库模式)：**
  - 将数据集中的“漏洞代码”和“成功 Exploit”存入向量数据库（如 Pinecone）。
  - 当 Agent 拿到一个待测合约时，先搜索数据集中最相似的案例，学习其攻击手法。
- **Few-Shot 提示工程 (样本模式)：**
  - 从数据集中挑选 3-5 个典型的漏洞利用案例（CVE/CWE）作为 Prompt 的上下文（Few-shot examples），指导 Agent 生成更精准的代码。
- **自动化验证 (评估模式)：**
  - 数据集通常包含“预期结果”。构建一个 **Assessor/Verifier** 节点，专门对比 Agent 跑出的结果与数据集中的 Ground Truth 是否一致。

工具合集：

环节	推荐工具	作用
多代理编排	**LangGraph**	控制“分析-尝试-验证”的闭环逻辑。
漏洞知识存储	**Pinecone /**	存储你的漏洞数据集，实现语义搜索增强。

	Milvus	
底层仿真实验室	**Foundry (Anvil)**	基于数据集提供的块高度，开启 Local Fork 进行实战演练。
静态引导分析	**Slither**	结合数据集特征，提供初始的静态分析结果。

基准测试

- 关注 Agent 的 **召回率 (Recall)**（数据集里的漏洞它找出了多少）和 **准确率 (Precision)**。
- 参考 **CVE-Bench** 的沙盒设计，确保 Agent 在容器化（Docker）环境中安全地执行数据集里的攻击代码，防止损坏主机系统。

系统构建

第一层：定义“共享状态” (The State Schema)

这是系统的“内存”，在 LangGraph 的各个 Agent 节点间流转。我们将原有的攻击性字段替换为审计专用字段。

```
from typing import TypedDict, List, Dict, Optional

class AuditGraphState(TypedDict):
    # --- 1. 上下文输入 (Context) ---
    contract_source: str        # 目标合约源码
    contract_abi: List          # 目标合约 ABI

    # --- 2. 静态分析结果 (Node 1 产出) ---
    # 识别出的角色 (如 Owner, Admin) 和修饰符 (Modifiers)
    defined_roles: List[str]
    # 待审计的敏感函数列表，包含函数名、签名及理论上的权限要求
    # 例: [{"name": "setFee", "signature": "0x...", "expected_ro
le": "Owner"}]
    sensitive_functions: List[Dict]

    # --- 3. 验证过程 (Node 2 & 3 交互) ---
    # 审计假说：Agent 怀疑哪里存在越权风险
    # 例: "Function 'initialize' is public and lacks 'initialize
r' modifier."
    audit_hypothesis: str
```

```
    # 最小验证用例 (PoC): Foundry 测试脚本，用于证明权限失效
    # 注意: 不再是 Exploit，而是 Evidence Code
    verification_poc: str

    # 执行轨迹: Anvil 的 trace 或 console.log 输出
    execution_trace: str

    # --- 4. 最终结论 (Output) ---
    retry_count: int            # 当前尝试迭代次数
    finding_confirmed: bool     # 是否确认存在漏洞 (True = 审计不通
过)
    audit_report: Dict          # 结构化的漏洞报告 (位置、类型、证据)`
```

第二层：设计“核心节点” (The Worker Nodes)

系统包含三个专注于审计工作的核心节点。

Node 1: 权限模式分析师 (Access Control Analyst)

- **职责**： 静态分析与模式识别。
- **输入**： `contract_source` + **RAG 向量库 (C5 数据集)**。
- **动作**：
  1. **角色提取**：识别合约中定义的 `onlyOwner`, `AccessControl`, `Auth` 等权限结构。
  2. **敏感点扫描**：找出所有改变合约状态（State-Changing）的关键函数（如 `mint`, `pause`, `upgradeTo`）。
  3. **模式匹配**：结合数据集中的漏洞模式（如“缺少初始化保护”），对比当前代码，标记出**约束缺失**的可疑函数。
- **输出**： 更新 State 中的 `sensitive_functions` 和 `audit_hypothesis`。

Node 2: 证据构建器 (Evidence Builder)

- **职责**： 编写最小验证单元测试 (Unit Test Generator)。
- **输入**： `sensitive_functions` + `audit_hypothesis`。
- **动作**：
  - 编写一个 **Foundry (.t.sol)** 测试脚本。
  - **核心逻辑设定**：
    - `Setup`: 部署目标合约。
    - `Actor`: 创建一个**未授权地址** (`address unauthorized = makeAddr("stranger");`)。

    - `Action`：使用 `vm.prank(unauthorized)` 模拟该地址调用目标敏感函数。
    - `Assertion`：**不做利润断言**，只做状态或 Revert 断言。
- **代码示例 (Prompt 目标)**：Solidity

```
function test_PrivilegeEscalation() public {
    address stranger = address(0x123);
    vm.prank(stranger);

    // 尝试调用只有管理员能调用的函数
    targetContract.setProtocolFee(100);

    // 审计标准：如果上述调用没有 Revert，且状态确实变了，则漏洞存在
    assertEq(targetContract.fee(), 100, "Access Control Failed: Unauthorized user changed fee");
}
```

- **输出**： 更新 State 中的 `verification_poc`。

Node 3: 动态验证器 (Dynamic Verifier)

- **职责**： 执行测试并解析结果 (The Ground Truth Checker)。
- **输入**： `verification_poc`。
- **动作**：
    1. 在 **Anvil** 本地分叉环境中运行 `forge test --json`。
    2. **解析日志**：
        - 如果测试通过 (`Status: PASS`)：说明未授权调用**成功执行**了（漏洞证实）。
        - 如果测试失败 (`Status: FAIL` 且原因是 `Revert: Ownable: caller is not the owner`)：说明权限控制**生效**了（安全）。
        - 如果测试失败 (编译错误或逻辑错误)：说明 PoC 写错了（需要重试）。
- **输出**： 更新 State 中的 `execution_trace` 和 `finding_confirmed`。

第三层：构建“控制流” (The Control Flow)

利用 LangGraph 的 `Conditional Edge` 实现从“假说”到“证据”的闭环。

路由逻辑 (Audit Router):

1. **分支 A：漏洞证实 (Vulnerability Confirmed)**

- **条件：** Node 3 返回 `PASS`（意味着 PoC 里的断言 `assertEq` 成功，即未授权操作被允许了）。
- **去向：** **END 节点**。
- **动作：** 生成结构化审计报告，包含“漏洞位置”、“修复建议”和“复现代码”。

2. **分支 B：修正证据 (Refine Evidence)**
   - **条件：** Node 3 返回 `FAIL`（可能是编译错误，或者 PoC 逻辑没写对导致被拦截）**且** `retry_count < MAX_RETRIES`。
   - **去向：** 回到 **Node 2 (证据构建器)**。
   - **动作：** 将报错信息（Trace）喂回给 LLM，让它修改测试脚本（例如：修正函数签名、调整 Prank 逻辑）。
3. **分支 C：排除风险 (Risk Dismissed)**
   - **条件：** Node 3 明确返回“权限拦截 Revert” **或** `retry_count >= MAX_RETRIES`。
   - **去向：** **END 节点**。
   - **动作：** 输出“该函数访问控制看似正常”或“无法生成有效证据”。

第四层：数据集的“特殊工位” (Dataset Integration)

数据集在本架构中扮演“教官”和“考官”的双重角色。

1. RAG 知识增强 (Input Side)

- **内容：** 存储 **C5 专项漏洞模式**（JSON 格式）。
  - *例：* `{ "pattern": "Initializer Unprotected", "code_snippet": "...", "fix": "Add initializer modifier" }`
- **集成方式：** Node 1 在分析时，先计算目标代码的 Embedding，去向量库检索最相似的 3 个历史漏洞模式，作为 Prompt 的上下文 (Few-Shot Context)。

2. 自动化评估/回归测试 (Output Side)

- **场景：** 系统开发与调优阶段。
- **流程：**
  1. 从数据集中抽取 50 个已知存在权限漏洞的合约（Ground Truth = Vulnerable）。
  2. 运行本 Agent 系统。
  3. **计算指标：**
     - **Recall (召回率)：** Agent 成功检出了多少个？（目标：不漏报）
     - **Proof Quality：** 生成的 PoC 是否真的能在 Anvil 中跑通？

4. **错题分析：** 针对漏报的案例（False Negatives），分析是 Node 1 没找出来，还是 Node 2 的 PoC 写错了。

总结架构图 (Text Representation)

```
graph TD
    Start[输入目标合约] --> Node1

    subgraph "Audit Loop"
        Node1[Node 1: 权限分析与RAG检索] --> StateUpdate1[更新 State: 敏感函数列表]
        StateUpdate1 --> Node2[Node 2: 构建审计PoC]
        Node2 --> Node3[Node 3: Anvil 动态验证]

        Node3 --> Check{验证结果判定?}
    end

    Check -- "PoC执行成功 (漏洞证实)" --> Report[生成审计报告]
    Check -- "PoC执行失败 (编译错/逻辑错)" --> Retry{重试 < 5次?}
    Check -- "明确被Revert拦截 (安全)" --> Safe[标记为安全/无漏洞]

    Retry -- Yes (带报错日志) --> Node2
    Retry -- No (超时) --> Safe

    Report --> End
    Safe --> End
```